

Lekcja

Temat: Projektowanie baz danych



Projektowanie baz danych to proces tworzenia logicznej i fizycznej struktury bazy danych, która efektywnie przechowuje, organizuje i zarządza danymi, spełniając wymagania konkretnej aplikacji lub systemu.

Kluczowym aspektem tego procesu jest świadomość, że **w bazie danych przechowujemy tylko niektóre informacje o świecie rzeczywistym**. Wybór właściwych wycinków rzeczywistości i dotyczących ich danych jest bardzo istotny — od niego zależy prawidłowe działanie bazy. Aby ten wybór był właściwy, należy wskazać informacje, które powinny być przechowywane w bazie danych, oraz określić ich strukturę.



Główne etapy projektowania

Cały proces projektowania bazy danych możemy podzielić na kilka etapów:



1. Planowanie bazy danych

- Analiza wymagań systemu i użytkowników
- Określenie, które **fragmenty rzeczywistości** mają być modelowane
- Identyfikacja **konkretnych danych** potrzebnych systemowi
- Definiowanie celów, zakresu i ograniczeń projektu
- Określenie przyszłych potrzeb roszszerzania bazy



2. Tworzenie modelu konceptualnego (diagramy ERD)

- Identyfikacja **encji** (tabel) reprezentujących wybrane byty rzeczywiste
- Określenie **atrybutów** (kolumn) opisujących te byty
- Definiowanie **relacji** między encjami

- Tworzenie diagramów ERD (Entity-Relationship Diagrams)
- **Selekcja** - co włączyć, co pominąć w modelu danych



3. Transformacja modelu konceptualnego na model relacyjny

- Przekształcenie encji na tabele
- Konwersja atrybutów na kolumny
- Zamiana relacji na klucze obce i tabele łączące
- Definiowanie typów danych dla kolumn
- Określenie ograniczeń (constraints)



4. Proces normalizacji bazy danych

- **1NF** (Pierwsza postać normalna): eliminacja powtarzających się grup
- **2NF**: eliminacja zależności częściowych od klucza
- **3NF**: eliminacja przechodnich zależności
- Ocena konieczności dalszych postaci normalnych (BCNF, 4NF, 5NF)
- Rozważenie celowej denormalizacji dla poprawy wydajności



5. Wybór struktur i określenie zasad dostępu do bazy danych

- Wybór silnika bazy danych w MySQL (InnoDB, MyISAM)
- Definiowanie indeksów dla optymalizacji zapytań
- Określenie zasad bezpieczeństwa i uprawnień
- Planowanie kopii zapasowych i recovery
- Optymalizacja struktur pod kątem wydajności



Podstawowe pojęcia w projektowaniu baz danych



Encja (Entity)

Encją jest każdy przedmiot, zjawisko, stan lub pojęcie, czyli każdy obiekt, który potrafimy odróżnić od innych obiektów (na przykład: osoba, samochód, książka, stan pogody).



Zbiór encji

Encje podobne do siebie (opisywane za pomocą podobnych parametrów) grupujemy w **zbiory encji**. W praktyce relacyjnej każdy zbiór encji staje się tabelą w bazie danych.

Przykład:

- Zbiór encji "Studenci" - zawiera poszczególne encje: student Jan Kowalski, student Anna Nowak, itd.
- Zbiór encji "Książki" - zawiera: "Pan Tadeusz", "Kamienie na szaniec", "Dzieci z Bullerbyn"

Wskazówka: Projektując bazę danych, należy precyzyjnie zdefiniować encje i określić parametry, przy użyciu których będą opisywane.



Atrybut (Attribute)

Atrybut to cecha opisująca encję. Encje mają określone cechy wynikające z ich natury. Cechy te nazywamy atrybutami.

Atrybuty encji stają się kolumnami w tabeli. Każdy atrybut reprezentuje jedną kolumnę w tabeli, która przechowuje wartości tego atrybutu dla poszczególnych encji.

Zestaw atrybutów, które określamy dla encji, zależy od potrzeb bazy danych. Nie wszystkie cechy encji muszą być przechowywane - tylko te istotne z punktu widzenia systemu.



Dziedzina (Domena)

Dziedzina to zbiór dopuszczalnych wartości atrybutu. Atrybuty encji mogą przyjmować różne wartości. Projektując bazę danych, możemy określić, jakie wartości może przyjmować dany atrybut. Zbiór wartości atrybutu nazywamy dziedziną (domeną).



Problem z projektowaniem bazy danych: Relacje wiele-do-wielu



Przykład problemu: System rezerwacji hotelu

Błędne podejście:

```
CREATE TABLE reservations (  
  reservation_id INT PRIMARY KEY,  
  guest_name VARCHAR(100),  
  room_number VARCHAR(10),  
  check_in DATE,  
  check_out DATE  
);
```

Problem: Jeden gość może mieć wiele rezerwacji, a w jednej rezerwacji może być wielu gości (rodzina, grupa). Jeśli wpisujemy wszystkich gości w jednym polu guest_name jako "Jan Kowalski, Anna Kowalska, dziecko Kowalskie", pojawiają się problemy:

1. **Redundancja danych** - ten sam gość powtarza się w wielu rezerwacjach
2. **Trudność wyszukiwania** - jak znaleźć wszystkie rezerwacje konkretnego gościa?
3. **Brak integralności** - nie ma kontroli nad poprawnością danych gości
4. **Problemy z modyfikacją** - zmiana danych gościa wymaga aktualizacji wielu rekordów



Rozwiązanie: Wydzielenie encji Gości i tabeli łączącej

Właściwe podejście:

```
-- Encja: Goście  
CREATE TABLE guests (  
  guest_id INT PRIMARY KEY AUTO_INCREMENT,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,  
  email VARCHAR(100) UNIQUE,  
  phone VARCHAR(20)  
);  
  
-- Encja: Rezerwacje  
CREATE TABLE reservations (  
  reservation_id INT PRIMARY KEY AUTO_INCREMENT,  
  check_in DATE NOT NULL,  
  check_out DATE NOT NULL,  
  status ENUM('confirmed', 'pending', 'cancelled')  
);  
  
-- Tabela łącząca dla relacji wiele-do-wielu  
CREATE TABLE reservation_guests (  
  reservation_id INT,  
  guest_id INT,  
  is_main_guest BOOLEAN DEFAULT FALSE, -- dodatkowy atrybut relacji  
  PRIMARY KEY (reservation_id, guest_id),
```

```
FOREIGN KEY (reservation_id) REFERENCES reservations(reservation_id),  
FOREIGN KEY (guest_id) REFERENCES guests(guest_id)  
);
```



Zalety tego rozwiązania:

1. **Eliminacja redundancji** - dane każdego gościa przechowywane raz
2. **Łatwe wyszukiwanie** - proste zapytania o rezerwacje danego gościa
3. **Integralność danych** - kontrola przez klucze obce
4. **Elastyczność** - możliwość dodawania dowolnej liczby gości do rezerwacji
5. **Łatwość modyfikacji** - zmiana danych gościa w jednym miejscu



Inne częste problemy i ich rozwiązania:

Problem 1: Mieszanie jednostek miary w jednej kolumnie

Błąd: price DECIMAL(10,2) bez określenia waluty

Rozwiązanie: Dodaj kolumnę currency VARCHAR(3) lub normalizuj do osobnej tabeli walut

Problem 2: Przechowywanie historii zmian w głównej tabeli

Błąd: Aktualne i historyczne dane mieszane w jednej tabeli

Rozwiązanie: Wydziel tabelę historii z timestampami i flagą is_current

Problem 3: Nieatomowe atrybuty (adres w jednym polu)

Błąd: address VARCHAR(200) zawierający ulicę, miasto, kod

Rozwiązanie: Podziel na osobne kolumny: street, city, postal_code, country

Zasada: "Jedna encja = jedna odpowiedzialność"

Najczęstszy błąd w projektowaniu to próba upchnięcia zbyt wielu konceptów w jednej encji/tabeli. Każda tabela powinna reprezentować **jeden jasno zdefiniowany byt** z rzeczywistości. Jeśli zauważysz, że:

- dane się powtarzają
- masz puste pola dla niektórych rekordów
- trudno jest modyfikować dane
- zapytania stają się skomplikowane



Tworzenie modelu konceptualnego (diagramy ERD)



Definicja modelu konceptualnego

Konceptualne projektowanie bazy danych to konstruowanie schematu danych niezależnego od wybranego modelu danych, docelowego systemu zarządzania bazą danych, programów użytkowych czy języka programowania. Jest to abstrakcyjna reprezentacja struktury danych skupiona na ich znaczeniu i relacjach, a nie na implementacji.



Diagramy ERD (Entity Relationship Diagram)

Do tworzenia modelu graficznego schematu bazy danych wykorzystywane są diagramy związków encji, z których najpopularniejsze są diagramy **ERD (ang. Entity Relationship Diagram)**. Pozwalają one na:

1. **Modelowanie struktur danych** oraz związków zachodzących między tymi strukturami
2. **Prawie bezpośrednie przekształcenie** diagramu w schemat relacyjny
3. **Analizę struktury** bazy danych na wysokim poziomie abstrakcji
4. **Dokumentację** tworzonego systemu baz danych

Na diagramy ERD składają się trzy rodzaje elementów:

- ✓ zbiory encji,
- ✓ atrybuty encji,
- ✓ związki zachodzące między encjami.



1. Zbiory encji (Entities)

Encja to reprezentacja obiektu przechowywanego w bazie danych. Graficzną reprezentacją encji jest najczęściej prostokąt.

Przykład:



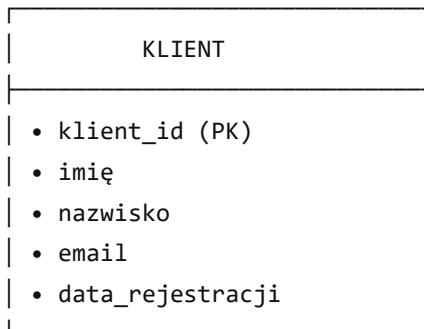
2. Atrybuty encji (Attributes)

Atrybut opisuje encję. Może on być liczbą, tekstem lub wartością logiczną. W relacyjnym modelu baz danych atrybut jest reprezentowany przez kolumnę tabeli.

Reprezentacje atrybutów:

- W notacji Chena: owal połączony z encją
- W notacji "pudełkowej": lista wewnątrz prostokąta encji

Przykład encji Klient z atrybutami:



3. Związki między encjami (Relationships)

Związek to powiązanie między dwoma zbiorami encji. Każdy związek ma dwa końce, do których są przypisane:

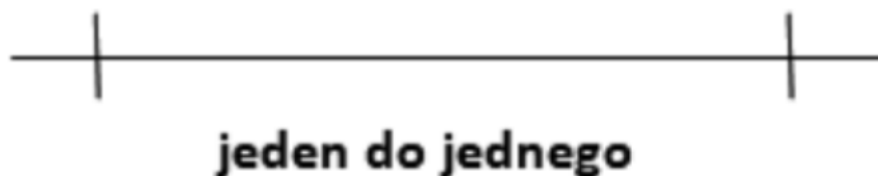


a) Stopień związku (Cardinality)

Określa, jakiego typu związek zachodzi między encjami:



1. Jeden do jednego (1:1)



Przykład: Pracownik ↔ Samochód służbowy

2.



Jeden do wielu (1:N)



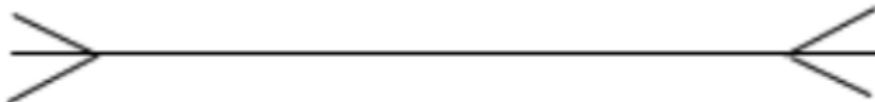
jeden do wielu

Przykład: Klient ↔ Zamówienia

3.



Wiele do wielu (N:M)



wiele do wielu

Przykład: Student ↔ Kursy

b)



Opcjonalność związku (Optionality)

Określa, czy związek jest opcjonalny, czy wymagany:

•



Wymagany (mandatory):

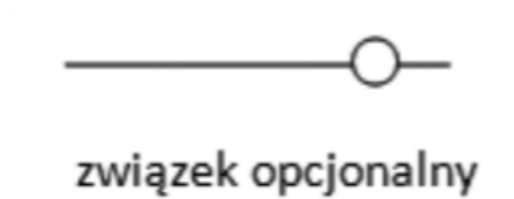


związek wymagany

•

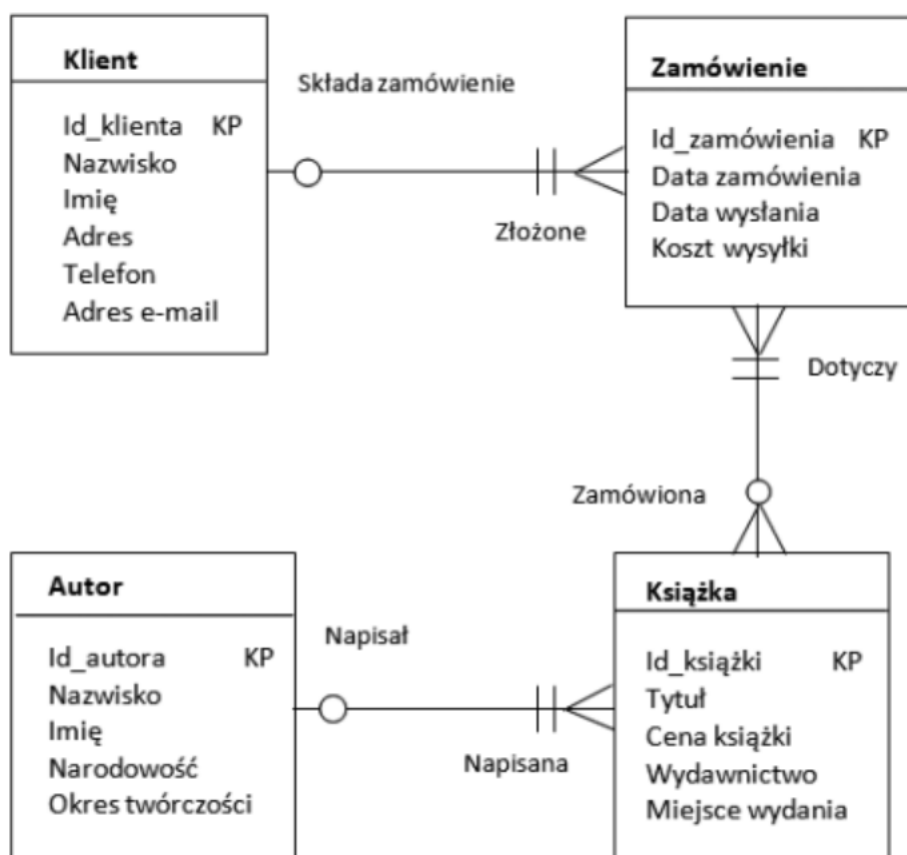


Opcjonalny (optional):



Diagramy ERD spotyka się w wielu różnych notacjach, na przykład: Martina, Bachmana, Chena, IDEF1X

Prosty przykład diagramu ERD w notacji Martina:



Lekcja

Temat: Typy danych w MySQL – przykłady

1. VARCHAR vs CHAR (haki z długością znaków)

```
CREATE TABLE test_text (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  v VARCHAR(10),  
  c CHAR(10)  
);
```

```
INSERT INTO test_text (v, c)  
VALUES ('ąźśćńóławq1', 'ąźśćń123456');
```

```
SELECT  
  v,  
  c,  
  CHAR_LENGTH(v) AS znaki_v,  
  LENGTH(v) AS bajty_v,  
  CHAR_LENGTH(c) AS znaki_c,  
  LENGTH(c) AS bajty_c,  
  OCTET_LENGTH(v) AS octet_v -- to samo co LENGTH  
FROM test_text;
```

2. TIME(3) – milisekundy

```
CREATE TABLE test_time (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  t1 TIME,  
  t2 TIME(3)  
);
```

```
INSERT INTO test_time (t1, t2)  
VALUES ('12:30:15.987', '12:30:15.987');
```

```
SELECT * FROM test_time;
```

3. FLOAT – błąd dokładności

```
CREATE TABLE test_float (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  a FLOAT  
);
```

```
INSERT INTO test_float (a)  
VALUES (0.1), (0.2);
```

```
INSERT INTO test_float (a)
VALUES (0.1234567), (2220.1234567);
```

TEST (klasyczny błąd)

```
SELECT SUM(a) FROM test_float;
```

4. POINT – geometria

```
CREATE TABLE miejsca (
  id INT PRIMARY KEY,
  nazwa VARCHAR(50),
  lokalizacja POINT NOT NULL
);
```

```
INSERT INTO miejsca VALUES
(1, 'Warszawa', POINT(52.2297, 21.0122)),
(2, 'Łódź', POINT(51.7592, 19.4550));
```

TEST (odległość „kartkowa”)

```
SELECT
  a.nazwa,
  b.nazwa,
  ST_Distance(a.lokalizacja, b.lokalizacja) AS dist
FROM miejsca a
JOIN miejsca b ON a.id < b.id;
```

5. „PROMIEŃ X”

TEST – znajdź „blisko Warszawy”

```
SELECT *
FROM miejsca
WHERE lokalizacja
  BETWEEN POINT(52.20, 21.00)
  AND POINT(52.30, 21.10);
```

6. INT vs UNSIGNED

```
CREATE TABLE test_int (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  normal INT,  
  positive_only INT UNSIGNED  
);
```

```
INSERT INTO test_int (normal, positive_only)  
VALUES (-10, 10);  
INSERT INTO test_int (normal, positive_only)  
VALUES (-10, -10);
```

```
SELECT * FROM test_int;
```

7. BOOLEAN (ukryty TINYINT)

```
CREATE TABLE test_bool (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  is_active BOOLEAN  
);
```

```
INSERT INTO test_bool (is_active)  
VALUES (1), (0), (TRUE), (FALSE);
```

```
SELECT * FROM test_bool;
```

8. JSON (nowoczesny SQL)

```
CREATE TABLE test_json (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  data JSON  
);
```

```
INSERT INTO test_json (data)  
VALUES ('{"name":"Ala","age":20}');
```

```
SELECT JSON_EXTRACT(data, '$.name') FROM test_json;
```

9. DECIMAL vs FLOAT/DOUBLE – nigdy nie używaj FLOAT do pieniędzy!

```

CREATE TABLE test_decimal (
  id INT PRIMARY KEY AUTO_INCREMENT,
  cena_float FLOAT,
  cena_double DOUBLE,
  cena_dec DECIMAL(10,4)
);

INSERT INTO test_decimal (cena_float, cena_double, cena_dec)
VALUES
(0.1, 0.1, 0.1),
(0.2, 0.2, 0.2);

SELECT
  cena_float,
  cena_double,
  cena_dec,
  cena_float + cena_double AS suma_float,
  cena_dec + cena_dec AS suma_dec
FROM test_decimal;

```

10. TIMESTAMP vs DATETIME – magiczne aktualizowanie i strefy czasowe

```

CREATE TABLE test_timestamp (
  id INT PRIMARY KEY AUTO_INCREMENT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  plain_ts TIMESTAMP NULL
);

```

```

INSERT INTO test_timestamp () VALUES ();

```

```

SELECT * FROM test_timestamp;

```

-- po 2 sekundach zaktualizuj rekord:

```

UPDATE test_timestamp
SET plain_ts = NOW();
SELECT * FROM test_timestamp;

```

updated_at zmienia się automatycznie przy każdym UPDATE.

created_at (DATETIME) nigdy się nie zmienia.

plain_ts bez DEFAULT zachowuje się inaczej w zależności od wersji MySQL.

11. ENUM – wygląda jak tekst, ale jest liczbą

```
CREATE TABLE test_enum (
  id INT PRIMARY KEY AUTO_INCREMENT,
  status ENUM('nowy', 'w_toku', 'zakończony', 'anulowany') NOT NULL
);
```

```
INSERT INTO test_enum (status) VALUES
('nowy'), ('w_toku'), ('zakończony');
```

```
SELECT
  status,
  status + 0 AS kod_liczbowy,    -- <-- haczyk!
  CASE WHEN status = 1 THEN 'nowy' ELSE status END AS test
FROM test_enum;
```

Haczyk: status + 0 zwraca 1, 2, 3... (kolejność w definicji ENUM).
Możesz nawet zrobić WHERE status = 2 i zadziała.

12. ST_Distance na SRID 0 vs 4326 – dystans w „stopniach” czy w metrach?

```
CREATE TABLE test_odleglosc (
  id INT PRIMARY KEY,
  nazwa VARCHAR(50),
  lokalizacja POINT NOT NULL -- MySQL 8.0+
);
```

```
INSERT INTO test_odleglosc VALUES
(1, 'Warszawa', POINT(52.2297, 21.0122)),
(2, 'Łódź', POINT(51.7592, 19.4550));
```

POINT(52.2297, 21.0122) = szerokość (lat), długość (lon)

Szerokość geograficzna (latitude, lat)

Określa, jak daleko na **północ** lub **południe** od równika się znajdujesz.

- Zakres: **-90° do +90°**
- 0° → równik
- Wartości **dodatnie (+)** lub oznaczenie **N** → półkula północna
- Wartości **ujemne (-)** lub oznaczenie **S** → półkula południowa

Przykład:

Warszawa ma około **52° N** → znajduje się na półkuli północnej, 52 stopnie na północ od równika.

Długość geograficzna (longitude, lon)

Określa, jak daleko na **wschód** lub **zachód** od południka zerowego (Greenwich) się znajdujesz.

- Zakres: **-180° do +180°**
- 0° → południk zerowy (Greenwich w Londynie)
- Wartości **dodatnie (+)** lub oznaczenie **E** → wschód
- Wartości **ujemne (-)** lub oznaczenie **W** → zachód

Przykład:

Warszawa ma około **21° E** → znajduje się 21 stopni na wschód od Greenwich.

SELECT

```
a.nazwa,  
b.nazwa,  
ST_Distance(a.lokalizacja, b.lokalizacja) AS dystans_stopnie,  
ST_Distance_Sphere(a.lokalizacja, b.lokalizacja) AS dystans_metrow
```

FROM test_odleglosc a

JOIN test_odleglosc b ON a.id < b.id;

Haczyk: Bez `ST_Distance_Sphere` dostajesz ~0.8 (stopnie), a nie ~130 km.

13. GENERATED COLUMN (kolumna generowana) – MySQL

5.7+

```
CREATE TABLE test_generated (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  imie VARCHAR(50),  
  nazwisko VARCHAR(50),  
  pelne_imie VARCHAR(101) AS (CONCAT(imie, ' ', nazwisko)) STORED,  
  inicjaly VARCHAR(10) AS (CONCAT(LEFT(imie,1), LEFT(nazwisko,1))) VIRTUAL  
);
```

INSERT INTO test_generated (imie, nazwisko) **VALUES** ('Jan', 'Kowalski');

SELECT * FROM test_generated;

Haczyk:

`pelne_imie` jest fizycznie zapisywane (STORED) → można na nim zrobić indeks.

`inicjaly` są obliczane na bieżąco (VIRTUAL) → nie zajmuje miejsca.

